

Design and Implementation of a Mini Compiler for a Domain Specific Language using Flex, Bison and C

Name: Raga Amirtha.A

Reg No: RA2311026050054

1. Introduction

A compiler is a system software that translates high-level programming language code into a lower-level representation such as machine code or intermediate code. This project focuses on the design and implementation of a mini compiler for a simple Domain Specific Language (DSL).

The compiler processes input code through multiple stages including lexical analysis, parsing, Abstract Syntax Tree (AST) construction, semantic analysis, and intermediate code generation.

2. Problem Statement

The objective of this project is to design and implement a compiler that can:

- Accept DSL input programs**
 - Analyze the syntax and structure**
 - Construct an internal representation (AST)**
 - Perform semantic checking**
 - Generate intermediate code (LLVM-like representation)**
-

3. System Architecture

Input Code



Lexical Analysis (Flex)



Parsing (Bison)



Abstract Syntax Tree



Semantic Analysis



Intermediate Code Generation

4. Tools and Technologies Used

- **Flex (Lex)** – for lexical analysis
 - **Bison (YACC)** – for parsing and grammar rules
 - **C Programming Language** – for implementation
 - **GCC Compiler** – for compilation
 - **WSL (Ubuntu)** – development environment
-

5. Modules Description

5.1 Lexical Analysis

The lexical analyzer is implemented using Flex. It reads the input DSL code and converts it into tokens such as:

- **Keywords (let, print)**
- **Identifiers**
- **Numbers**
- **Operators**

5.2 Parsing

The parser is implemented using Bison. It checks whether the sequence of tokens follows the defined grammar rules. If valid, it constructs a parse structure.

Example rule:

`decl → let id = expr ;`

5.3 Abstract Syntax Tree (AST)

The AST represents the hierarchical structure of the program. Each node corresponds to a language construct such as:

- Declaration
- Assignment
- Binary operations
- Print statements

The AST simplifies further processing like semantic analysis and code generation.

5.4 Semantic Analysis

Semantic analysis ensures that the program is logically correct.

In this project:

- Basic validation is performed
 - No semantic errors are detected in valid input programs
-

5.5 Intermediate Code Generation

The compiler generates intermediate code in a simplified 3-address code format:

Example:

`t1 = y * 2`

`t2 = x + t1`

`z = t2`

`print z`

Additionally, LLVM-like IR generation is supported to represent the program in a lower-level form.

6. Sample Input

```
let x = 10;  
let y = 20;  
let z = x + y * 2;  
print z;
```

7. Sample Output

Parsing Successful!

Abstract Syntax Tree:

```
DECL: x  
  NUM: 10  
DECL: y  
  NUM: 20  
DECL: z  
  BINOP: +  
    ID: x  
    BINOP: *  
      ID: y  
      NUM: 2  
PRINT  
  ID: z
```

Semantic Analysis:

No errors found

Intermediate Code:

t1 = y * 2

t2 = x + t1

z = t2

print z

8. Results

The compiler successfully:

- **Tokenizes input using Flex**
 - **Parses syntax using Bison**
 - **Builds AST using C structures**
 - **Performs semantic checks**
 - **Generates intermediate code**
-

9. Conclusion

This project demonstrates the complete working of a compiler pipeline. It provides a clear understanding of how high-level code is transformed step-by-step into a structured intermediate representation.

The system can be further extended to generate optimized code or integrate with LLVM for full compilation.

10. Future Enhancements

- **Full LLVM IR generation and optimization**
 - **Support for control structures (loops, conditions)**
 - **Error handling for semantic violations**
 - **Code optimization techniques**
-